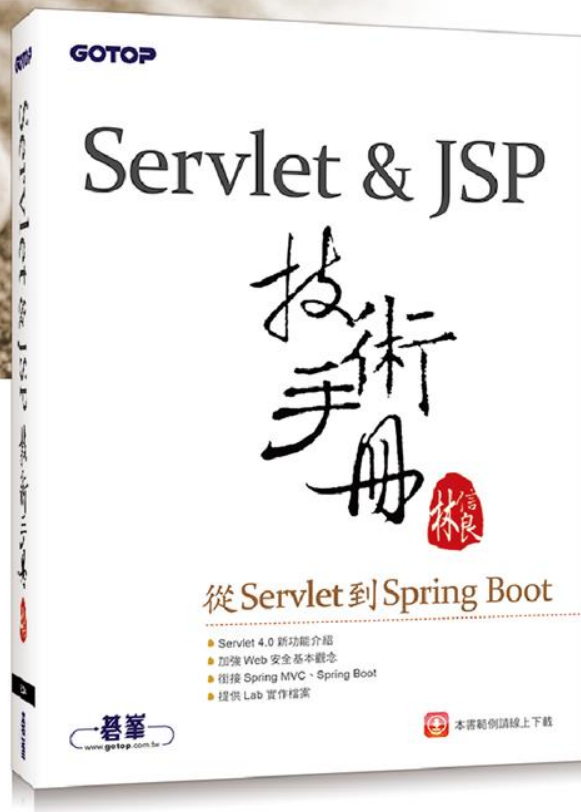




Servlet & JSP

技術手冊

從Servlet到Spring Boot

- Servlet 4.0 新功能介紹
- 加強 Web 安全基本觀念
- 銜接 Spring MVC、Spring Boot
- 提供 Lab 實作檔案

- Servlet進階API、過濾器與傾聽器



學習目標

- 了解Servlet生命週期
- 使用ServletConfig與ServletContext
- 使用PushBuilder
- 各種傾聽器的使用
- 繼承HttpFilter實作過濾器

Servlet 介面

- `init()`
- `service()`
- `destroy()`

ServletConfig介面



HTTP伺服器

Web 容器

<<create>>

<<create>>

config
:ServletConfig

init(config)

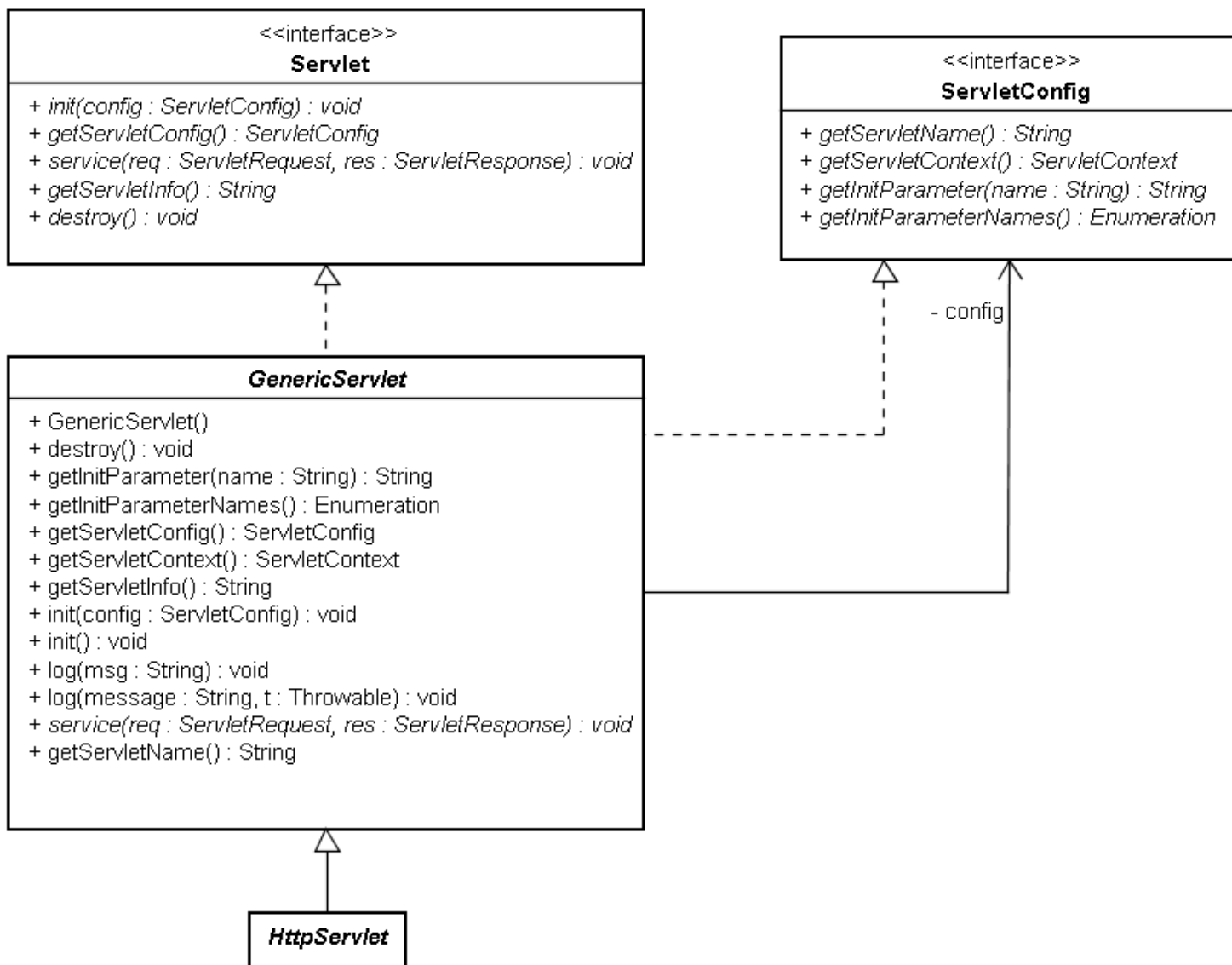
```
<servlet>
  <servlet-name>AddBookmark</servlet-name>
  <servlet-class>cc.openhome.controller.AddBookmark</servlet-class>
  <init-param>
    <param-name>SUCCESS</param-name>
    <param-value>success.view</param-value>
  </init-param>
</servlet>
```

destroy()

:HttpServlet

X

Servlet類別架構圖



GenericServlet類別

- 將初始Servlet呼叫init()方法傳入的ServletConfig封裝起來

```
private transient ServletConfig config;
public void init(ServletConfig config) throws ServletException {
    this.config = config;
    this.init();
}
public void init() throws ServletException {
}
```

- 有一些初始時所要執行的動作，可以重新定義這個無參數的init()方法

GenericServlet類別

- 包括了Servlet與ServletConfig定義方法的簡單實作

```
public ServletConfig getServletConfig() {  
    return config;  
}  
  
public String getInitParameter(String name) {  
    return getServletConfig().getInitParameter(name);  
}  
  
public Enumeration getInitParameterNames() {  
    return getServletConfig().getInitParameterNames();  
}  
  
public ServletContext getServletContext() {  
    return getServletConfig().getServletContext();  
}
```

使用ServletConfig

- 使用標註設定個別Servlet的初始參數

```
...
@WebServlet(name="ServletConfigDemo", urlPatterns={"/conf"},
            initParams={
                @WebInitParam(name = "PARAM1", value = "VALUE1"),
                @WebInitParam(name = "PARAM2", value = "VALUE2")
            }
)
public class ServletConfigDemo extends HttpServlet {
    private String PARAM1;
    private String PARAM2;
    public void init() throws ServletException {
        PARAM1 = getServletConfig().getInitParameter("PARAM1");
        PARAM2 = getServletConfig().getInitParameter("PARAM2");
    }
    ....
}
```

使用ServletConfig

- 在web.xml中設定個別Servlet的初始參數

```
...  
<servlet>  
    <servlet-name>ServletConfigDemo</servlet-name>  
    <servlet-class>cc.openhome.ServletConfigDemo</servlet-class>  
    <init-param>  
        <param-name>PARAM1</param-name>  
        <param-value>VALUE1</param-value>  
    </init-param>  
    <init-param>  
        <param-name>PARAM2</param-name>  
        <param-value>VALUE2</param-value>  
    </init-param>  
</servlet>  
...
```

使用ServletConfig

- GenericServlet定義了一些方法，將ServletConfig封裝起來，便於取得設定資訊

```
...
@WebServlet(name="ServletConfigDemo", urlPatterns={"/conf"},
            initParams={
                @WebInitParam(name = "PARAM1", value = "VALUE1"),
                @WebInitParam(name = "PARAM2", value = "VALUE2")
            }
)
public class AddMessage extends HttpServlet {
    private String PARAM1;
    private String PARAM2;
    public void init() throws ServletException {
        PARAM1 = getInitParameter("PARAM1");
        PARAM2 = getInitParameter("PARAM2");
    }
    ...
}
```

```
@WebServlet(  
    name="Login", ← ❶ 設定 Servlet 名稱  
    urlPatterns = {"/login"},  
    initParams = {  
        @WebInitParam(name = "SUCCESS", value = "success.view"),  
        @WebInitParam(name = "ERROR", value = "error.view")  
    }  
)  
public class Login extends HttpServlet {  
    private String SUCCESS_PATH;  
    private String ERROR_PATH;  
  
    @Override  
    public void init() throws ServletException {  
        SUCCESS_PATH = getInitParameter("SUCCESS");  
        ERROR_PATH = getInitParameter("ERROR");  
    }  
  
    @Override  
    protected void doPost(  
        HttpServletRequest request, HttpServletResponse response)  
        throws ServletException, IOException {  
        response.setContentType("text/html;charset=UTF-8");  
        String name = request.getParameter("name");  
        String passwd = request.getParameter("passwd");  
        String path = login(name, passwd) ? SUCCESS_PATH : ERROR_PATH;  
        response.sendRedirect(path);  
    }  
  
    private boolean login(String name, String passwd) {  
        return "caterpillar".equals(name) && "123456".equals(passwd);  
    }  
}
```

❷ 設定初始參數

❸ 取得初始參數

使用ServletConfig

- 如果web.xml中的設定要覆蓋標註設定，
<servlet-name>的設定必須與
@WebServlet的name屬性相同
- 如果不設定name屬性，預設是類別完整名稱

使用ServletConfig

```
<servlet>
  <servlet-name>Login</servlet-name> ← 注意 Servlet 名稱
  <servlet-class>cc.openhome.Login</servlet-class>
  <init-param>
    <param-name>SUCCESS</param-name>
    <param-value>success.html</param-value>
  </init-param>
  <init-param>
    <param-name>ERROR</param-name>
    <param-value>error.html</param-value>
  </init-param>
</servlet>
```

ServletContext介面

- 定義了運行Servlet的應用程式環境的一些行為與觀點
- 整個Web應用程式載入Web容器之後，容器會生成一個ServletContext物件作為整個應用程式的代表
- 透過ServletConfig的
`getServletContext()` 方法就可以取得

使用ServletContext

- `getRequestDispatcher()`

```
context.getRequestDispatcher("/pages/some.jsp")  
    .forward(request, response);
```

- `getResourcePaths()`

```
getServletContext().getResourcePaths("/")  
    .forEach(path -> out.println(path));
```

```
/welcome.html  
/catalog/  
/catalog/index.html  
/catalog/products.html  
/customer/  
/customer/login.jsp  
/WEB-INF/  
/WEB-INF/web.xml  
/WEB-INF/classes/com.acme.OrderServlet.class
```

使用ServletContext

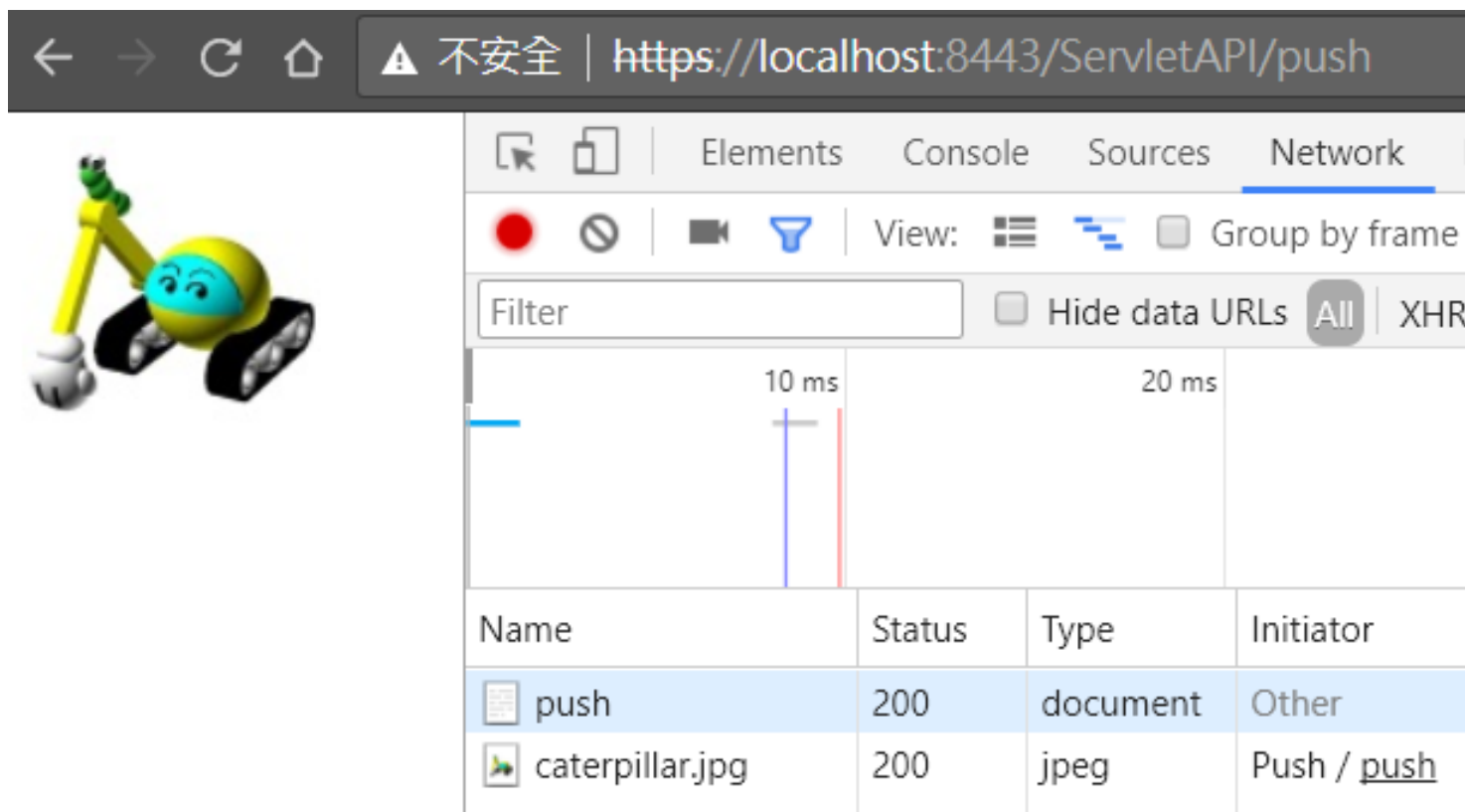
- `getResourceAsStream()`
 - 在3.3.3節中有個讀取PDF的範例

使用PushBuilder

- Servlet 4.0規範中制訂了對HTTP/2的支援，在伺服器推送上，提供了PushBuilder

```
Optional.ofNullable(request.newPushBuilder())
    .ifPresent(pushBuilder -> {
        pushBuilder.path("avatar/caterpillar.jpg")
            .addHeader("Content-Type", "image/jpeg")
            .push();
    });
```

使用PushBuilder



The screenshot shows a web browser window with the address bar displaying `https://localhost:8443/ServletAPI/push`. The page content includes a cartoon caterpillar character. The Network tab in the developer tools is open, showing a list of requests:

Name	Status	Type	Initiator
push	200	document	Other
caterpillar.jpg	200	jpeg	Push / <u>push</u>

ServletContextListener

- 想得知何時Web應用程式已經初始化或即將結束銷毀

```
package javax.servlet;  
import java.util.EventListener;  
public interface ServletContextListener extends EventListener {  
    public default void contextInitialized(ServletContextEvent sce) {}  
    public default void contextDestroyed(ServletContextEvent sce) {}  
}
```

ServletContextListener

```
@WebListener ← ❶ 使用@WebListener 標註
public class ServletContextAttributes implements ServletContextListener {
    public void contextInitialized(ServletContextEvent sce) {
        ServletContext context = sce.getServletContext(); ← ❸ 取得 ServletContext
        String avatar = context.getInitParameter("AVATAR"); ← ❹ 取得初始參數
        context.setAttribute("avatar", avatar); ← ❺ 設定 ServletContext 屬性
    }
}
```

...

```
<context-param>
    <param-name>AVATAR</param-name>
    <param-value>/avatars</param-value>
</context-param>
```

...

ServletContextListener

- 在Servlet 3.0之前，
ServletContextListener實作類別，必須在web.xml中設定

...

```
<listener>  
    <listener-class>cc.openhome.ContextParameterReader</listener-class>  
</listener>
```

...

ServletContextListener

- 有些應用程式的設定，必須在Web應用程式初始時進行

```
@WebListener()  
public class CookieConfig implements ServletContextListener {  
    @Override  
    public void contextInitialized(ServletContextEvent sce) {  
        ServletContext context = sce.getServletContext();  
        context.getSessionCookieConfig()  
            .setName("caterpillar-sessionId");  
    }  
}
```

ServletContextListener

- 在應用程式初始化時，也可以實作 `ServletContextListener` 進行 `Servlet`、過濾器等的建立、設定與註冊
- 好處是給予 `Servlet`、過濾器等更多設定上的彈性

```
public class Avatar2 extends HttpServlet {  
    private final Path AVATAR_DIR; ← ❶ final 值域  
  
    public Avatar2(Path AVATAR_DIR) {  
        this.AVATAR_DIR = AVATAR_DIR; ← ❷ 透過建構式設定  
    }  
}
```

ServletContextListener

```
@WebListener
public class Avatar2Initializer implements ServletContextListener {
    @Override
    public void contextInitialized(ServletContextEvent sce) {
        ServletContext context = sce.getServletContext();
        String AVATAR = context.getInitParameter("AVATAR");
        ServletRegistration.Dynamic servlet =
            context.addServlet(
                "Avatar2",          ← ❶ 設定 Servlet 的名稱
                new Avatar2(Paths.get(AVATAR)) ← ❷ 建立 Servlet 實例
            );
        servlet.setLoadOnStartup(1);
        servlet.addMapping("/avatar2"); ← ❸ 設定 URI 模式
    }
}
```

ServletContextAttributeListener

```
package javax.servlet;
import java.util.EventListener;
public interface ServletContextAttributeListener extends EventListener{
    public default void attributeAdded(ServletContextAttributeEvent scae) {}
    public default void attributeRemoved(ServletContextAttributeEvent scae) {}
    public default void attributeReplaced(ServletContextAttributeEvent scae) {}
}
```

```
...
@WebListener()
public class SomeContextAttrListener
    implements ServletContextAttributeListener {
    ...
}
```

```
...
<listener>
    <listener-class>cc.openhome.SomeContextAttrListener</listener-class>
</listener>
```

HttpSessionListener

```
package javax.servlet.http;
import java.util.EventListener;
public interface HttpSessionListener extends EventListener {
    public default void sessionCreated(HttpSessionEvent se) {}
    public default void sessionDestroyed(HttpSessionEvent se) {}
}

@WebListener()
public class LoginTokenRemover implements HttpSessionListener {
    @Override
    public void sessionDestroyed(HttpSessionEvent se) {
        HttpSession session = se.getSession();
        String user = session.getAttribute("login");
        // 修改資料庫欄位為登出狀態
    }
}
```

HttpSessionListener

```
...  
    <listener>  
        <listener-class>cc.openhome.LoginTokenRemover</listener-class>  
    </listener>  
...
```

HttpSessionListener

```
@WebListener
public class OnlineUsers implements HttpSessionListener {
    public static int counter;

    @Override
    public void sessionCreated(HttpSessionEvent se) {
        OnlineUsers.counter++;
    }

    @Override
    public void sessionDestroyed(HttpSessionEvent se) {
        OnlineUsers.counter--;
    }
}
```

```
out.printf("<h1>目前線上人數  %d 人</h1>", OnlineUsers.counter);
```

HttpSessionAttributeListener

```
package javax.servlet.http;
import java.util.EventListener;
public interface HttpSessionAttributeListener extends EventListener {
    public default void attributeAdded(HttpSessionBindingEvent se) {}
    public default void attributeRemoved(HttpSessionBindingEvent se) {}
    public default void attributeReplaced(HttpSessionBindingEvent se) {}
}
```

HttpSessionBindingListener

- 讓即將加入HttpSession的屬性物件實作，不需註釋或在web.xml中設定

```
package javax.servlet.http;
import java.util.EventListener;
public interface HttpSessionBindingListener extends EventListener {
    public default void valueBound(HttpSessionBindingEvent event) {}
    public default void valueUnbound(HttpSessionBindingEvent event) {}
}
```

HttpSessionAttributeListener

```
public class User implements HttpSessionBindingListener {  
    private String name;  
    private String data;  
  
    public User(String name) {  
        this.name = name;  
    }  
  
    public void valueBound(HttpSessionBindingEvent event) {  
        this.data = name + " 來自資料庫的資料...";  
    }  
  
    public String getData() {  
        return data;  
    }  
    public String getName() {  
        return name;  
    }  
}
```

HttpSessionIdListener

- Servlet 3.1新增
- HttpSession的Session ID發生變化時，就會呼叫sessionIdChanged()方法

請求相關的傾聽器

- `ServletRequestListener`
- `ServletRequestAttributeListener`
- `AsyncListener`
- `ReadListener`

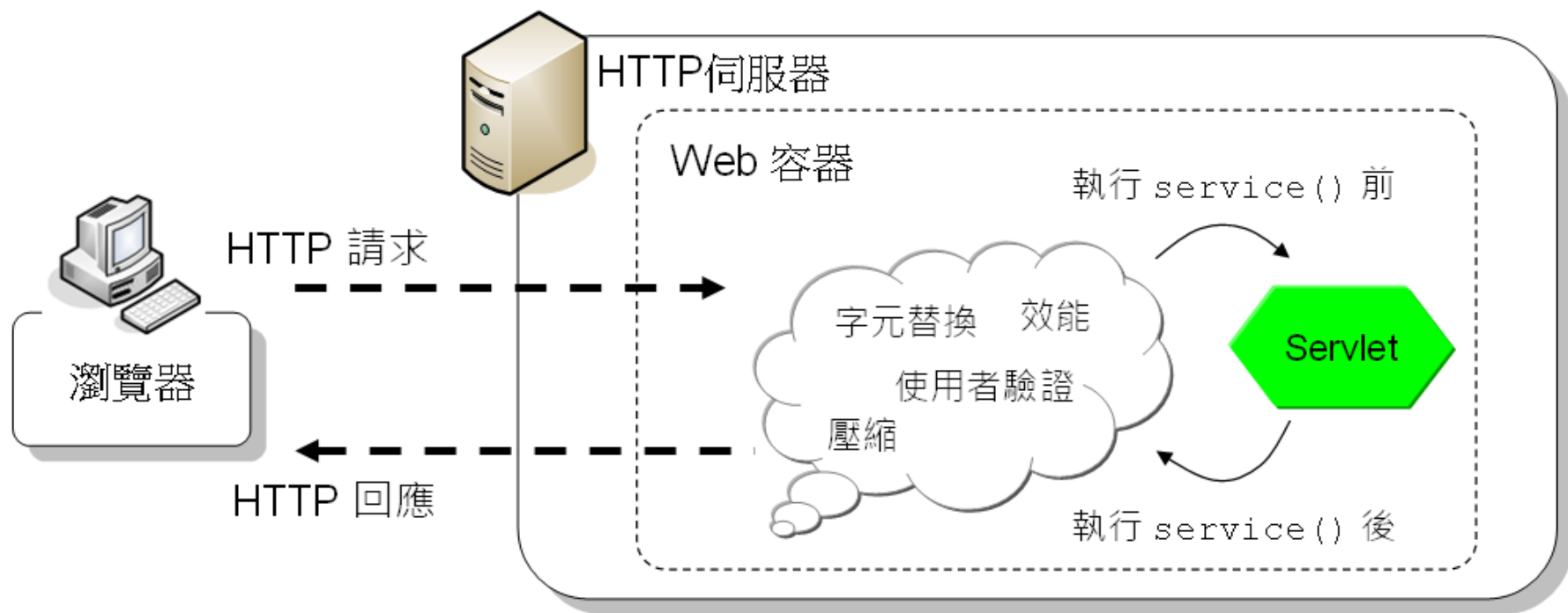
過濾器的概念

- 針對所有的Servlet，產品經理想要了解從請求到回應之間的時間差。
- 針對某些特定的頁面，客戶希望只有特定幾個使用者才可以瀏覽。
- 基於安全性的考量，使用者輸入的特定字元必須過濾並替換為無害的字元。
- 請求與回應的編碼從Big5改用UTF-8。
- ...

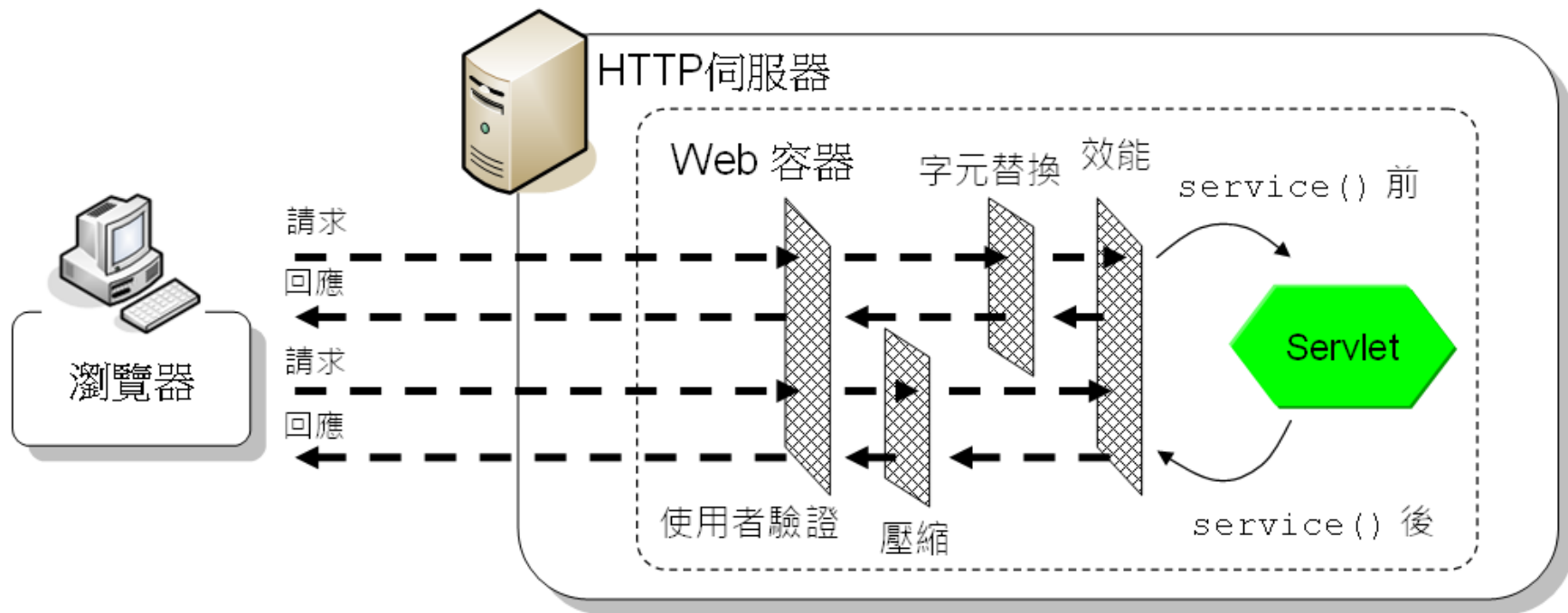
過濾器的概念

- 執行Servlet的`service()`方法「前」，記錄超始時間，Servlet的`service()`方法執行「後」，記錄結束時間並計算時間差。
- 執行Servlet的`service()`方法「前」，驗證是否為允許的使用者。
- 執行Servlet的`service()`方法「前」，對請求參數進行字元過濾與替換。
- 執行Servlet的`service()`方法「前」，對請求與回應物件設定編碼。
- ...

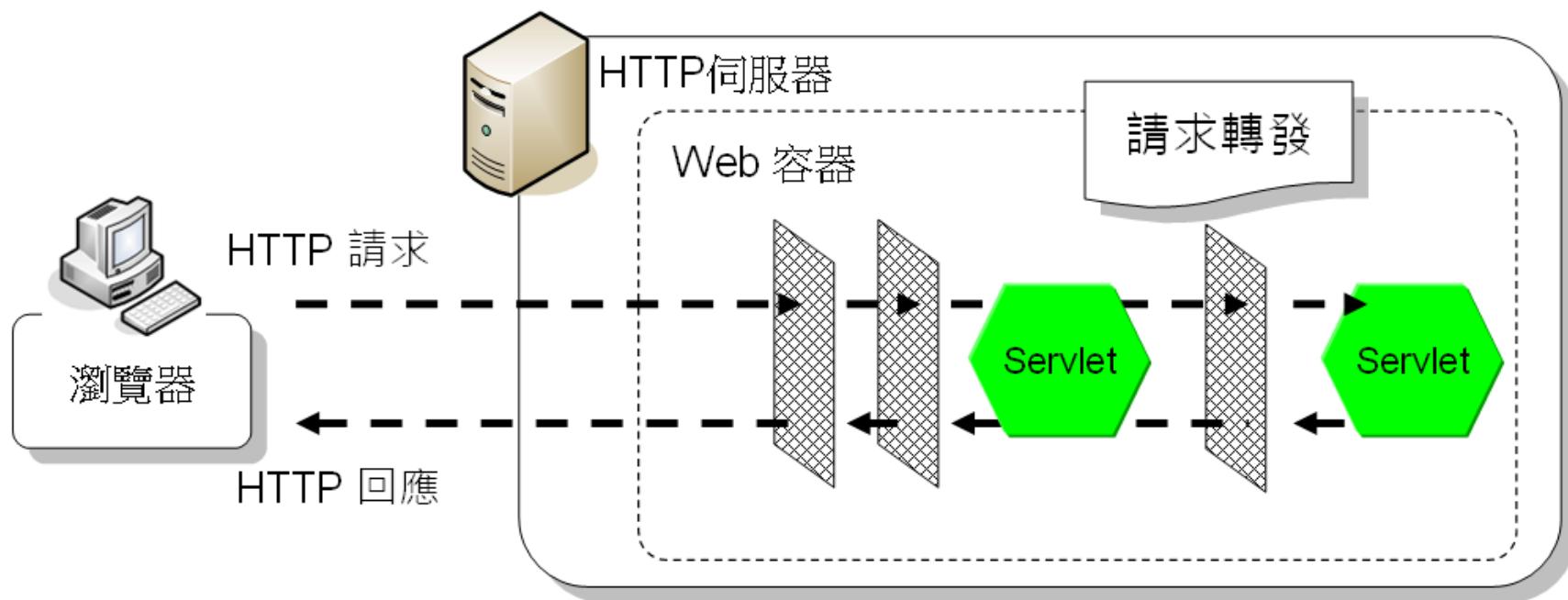
過濾器的概念



過濾器的概念



過濾器的概念



實作與設定過濾器

```
package javax.servlet;
import java.io.IOException;
public interface Filter {
    public default void init(FilterConfig filterConfig) throws ServletException {}

    public void doFilter(ServletRequest request, ServletResponse response,
        FilterChain chain) throws IOException, ServletException;

    public default void destroy() {}
}
```

實作與設定過濾器

- FilterChain的doFilter() 實作概念

```
Filter filter = filterIterator.next();
if(filter != null) {
    filter.doFilter(request, response, this);
}
else {
    targetServlet.service(request, response);
}
```

```
// service() 前置處理
chain.doFilter(request, response);
// service() 後置處理
```

GenericFilter與HttpFilter

- Servlet 4.0新增

```
public abstract class GenericFilter
    implements Filter, FilterConfig, Serializable {
    private static final long serialVersionUID = 1L;

    private volatile FilterConfig filterConfig;

    @Override
    public String getInitParameter(String name) {
        return getFilterConfig().getInitParameter(name);
    }

    @Override
    public Enumeration<String> getInitParameterNames() {
        return getFilterConfig().getInitParameterNames();
    }

    public FilterConfig getFilterConfig() {
        return filterConfig;
    }
}
```

```
@Override
public ServletContext getServletContext() {
    return getFilterConfig().getServletContext();
}

@Override
public void init(FilterConfig filterConfig) throws ServletException {
    this.filterConfig = filterConfig;
    init();
}

public void init() throws ServletException {

}

@Override
public String getFilterName() {
    return getFilterConfig().getFilterName();
}
}
```

```
public abstract class HttpFilter extends GenericFilter {
    private static final long serialVersionUID = 1L;

    @Override
    public void doFilter(
        ServletRequest request, ServletResponse response, FilterChain chain)
        throws IOException, ServletException {
        if (!(request instanceof HttpServletRequest)) {
            throw new ServletException(request + " not HttpServletRequest");
        }
        if (!(response instanceof HttpServletResponse)) {
            throw new ServletException(request + " not HttpServletResponse");
        }
        doFilter(
            (HttpServletRequest) request,
            (HttpServletResponse) response,
            chain);
    }

    protected void doFilter(
        HttpServletRequest request, HttpServletResponse response,
        FilterChain chain) throws IOException, ServletException {
        chain.doFilter(request, response);
    }
}
```

```
@WebFilter("/*") ← ❶ 使用@WebFilter 標註
public class TimeIt extends HttpFilter { ← ❷ 繼承 HttpFilter
    @Override
    protected void doFilter(
        HttpServletRequest request, HttpServletResponse response, FilterChain chain)
        throws IOException, ServletException {
        long begin = current();

        chain.doFilter(request, response);

        getServletContext().log(
            String.format("Request process in %d milliseconds", current() - begin)
        );
    }

    private long current() {
        return System.currentTimeMillis();
    }
}
```

實作與設定過濾器

...

```
<filter>
    <filter-name>TimeIt</filter-name>
    <filter-class>cc.openhome.TimeIt</filter-class>
</filter>
<filter-mapping>
    <filter-name>TimeIt</filter-name>
    <url-pattern>/*</url-pattern>
</filter-mapping>
```

...

實作與設定過濾器

- 在過濾器的請求套用上，除了指定URL模式之外，也可以指定Servlet名稱，這可以透過@WebServlet的**servletNames**來設定

```
@WebFilter(filterName="performance", servletNames={"SomeServlet"})
```

```
...
```

```
<filter-mapping>
```

```
<filter-name>performance</filter-name>
```

```
<servlet-name>SomeServlet</servlet-name>
```

```
</filter-mapping>
```

```
...
```

初始參數設定

```
...
@WebFilter(
    filterName="performance",
    urlPatterns={"/*"}, servletNames={},
    initParams={
        @WebInitParam(name = "PARAM1", value = "VALUE1"),
        @WebInitParam(name = "PARAM2", value = "VALUE2")
    }
)
public class PerformanceFilter implements Filter {
    private String PARAM1;
    private String PARAM2;

    @Override
    public void init(FilterConfig config) throws ServletException {
        PARAM1 = config.getInitParameter("PARAM1");
        PARAM2 = config.getInitParameter("PARAM2");
    }
    ...
}
```

...

```
<filter>
  <filter-name>PerformanceFilter</filter-name>
  <filter-class>cc.openhome.PerformanceFilter</filter-class>
  <init-param>
    <param-name>PARAM1</param-name>
    <param-value>VALUE1</param-value>
  </init-param>
  <init-param>
    <param-name>PARAM2</param-name>
    <param-value>VALUE2</param-value>
  </init-param>
</filter>
```

...

觸發過濾器的時機

- 預設是瀏覽器直接發出請求

```
@WebFilter(  
    filterName="some",  
    urlPatterns={"/some"},  
    dispatcherTypes={  
        DispatcherType.FORWARD,  
        DispatcherType.INCLUDE,  
        DispatcherType.REQUEST,  
        DispatcherType.ERROR,  
        DispatcherType.ASYNC  
    }  
)
```

...

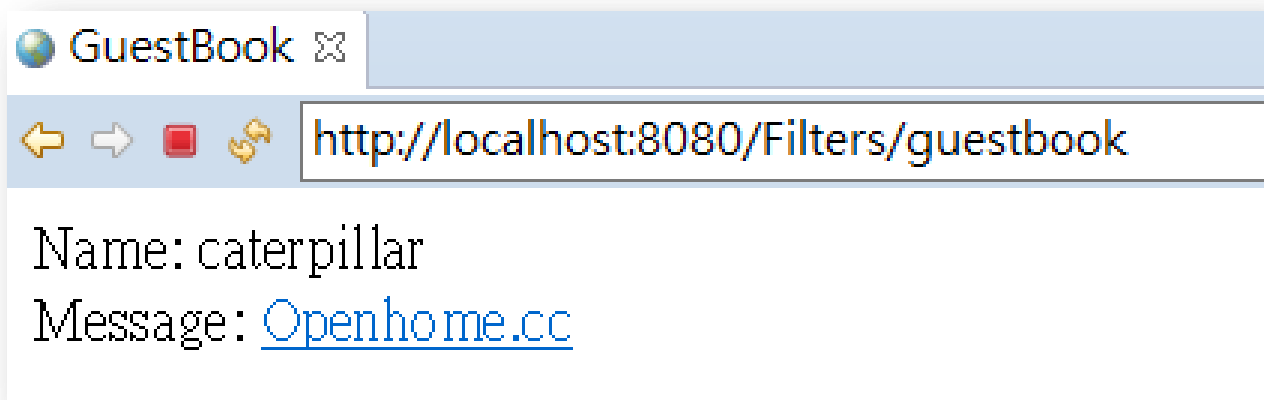
```
<filter-mapping>
  <filter-name>SomeFilter</filter-name>
  <servlet-name>*.do</servlet-name>
  <dispatcher>REQUEST</dispatcher>
  <dispatcher>FORWARD</dispatcher>
  <dispatcher>INCLUDE</dispatcher>
  <dispatcher>ERROR</dispatcher>
  <dispatcher>ASYNC</dispatcher>
</filter-mapping>
```

...

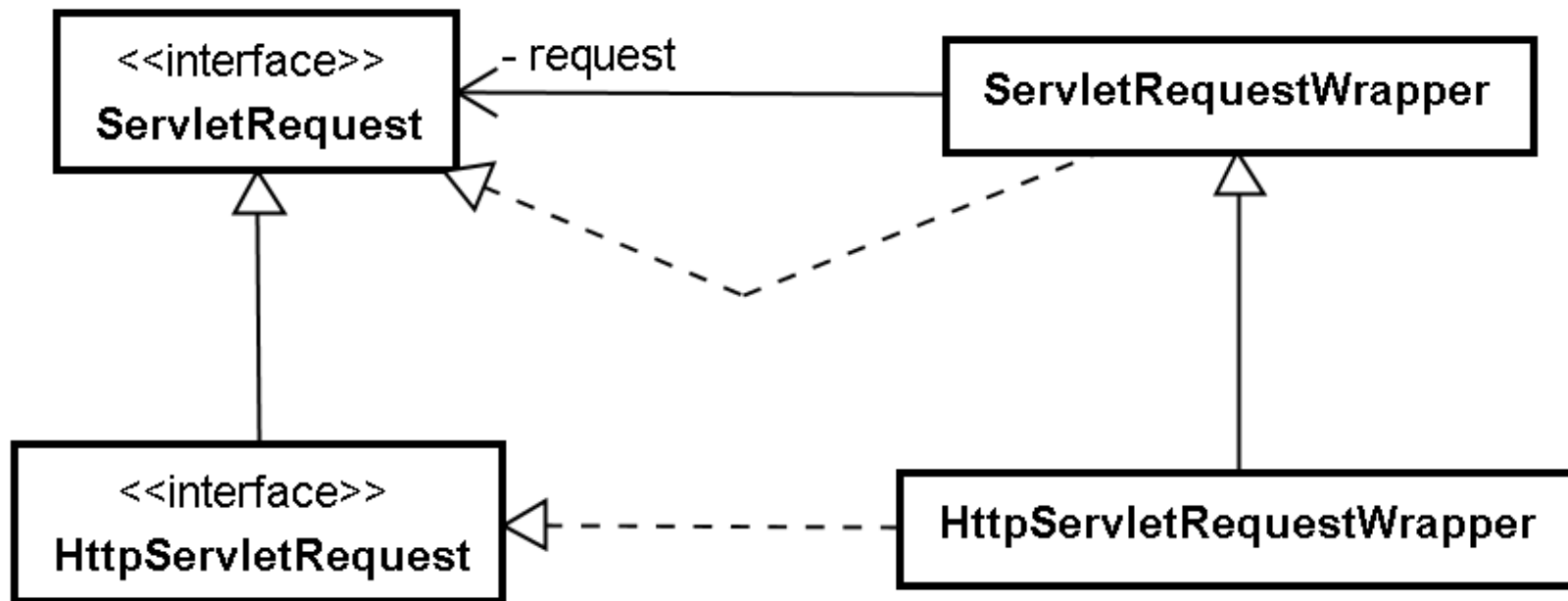
請求包裹器

- 實作字元替換過濾器

`<a href="http://openhome.cc">OpenHome.cc</a>`



請求包裹器

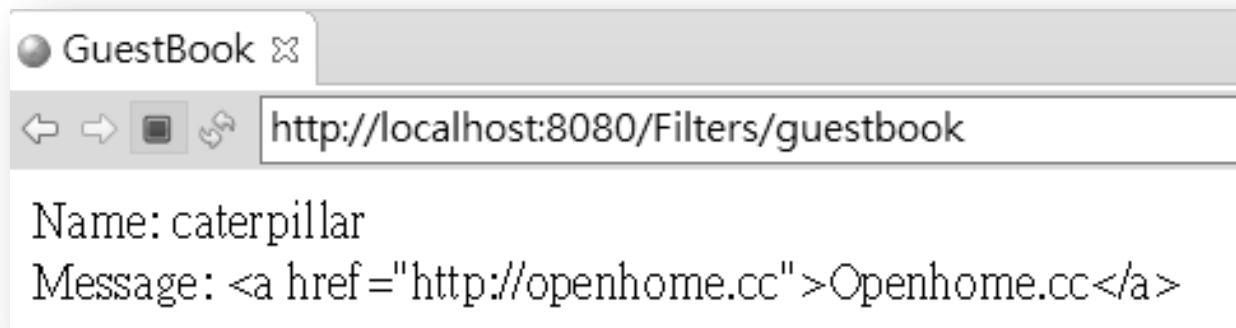


```
import org.owasp.encoder.Encode;

public class EncoderWrapper extends HttpServletRequestWrapper {
    public EncoderWrapper(HttpServletRequest request) {
        super(request); ← ❷ 必須呼叫父類別建構式，傳入 HttpServletRequest 實例
    }

    @Override
    public String getParameter(String name) { ← ❸ 重新定義 getParameter() 方法
        return Optional.ofNullable(getRequest().getParameter(name))
            .map(Encode::forHtml) ← ❹ 將取得的請求參數值進
            .orElse(null);          行字元替換
    }
}
```

```
@WebFilter("/*")
public class Encoder extends HttpFilter {
    public void doFilter(HttpServletRequest request,
        HttpServletResponse response, FilterChain chain)
        throws IOException, ServletException {
        ↓ 將原請求物件包裹在 EncoderWrapper 之中
        chain.doFilter(new EncoderWrapper(request), response);
    }
}
```



Openhome.cc

```
@WebFilter(  
    urlPatterns = { "/"* } ,  
    initParams = { @WebInitParam(name = "ENCODING", value = "UTF-8") }  
)  
public class Encoding extends HttpFilter {  
    public void doFilter(  
        HttpServletRequest request, HttpServletResponse response, FilterChain chain)  
        throws IOException, ServletException {  
        String encoding = getInitParameter("ENCODING");  
        request.setCharacterEncoding(encoding);  
        response.setCharacterEncoding(encoding);  
    }  
}
```

```
public class EncodingWrapper extends HttpServletRequestWrapper {
    private String ENCODING;

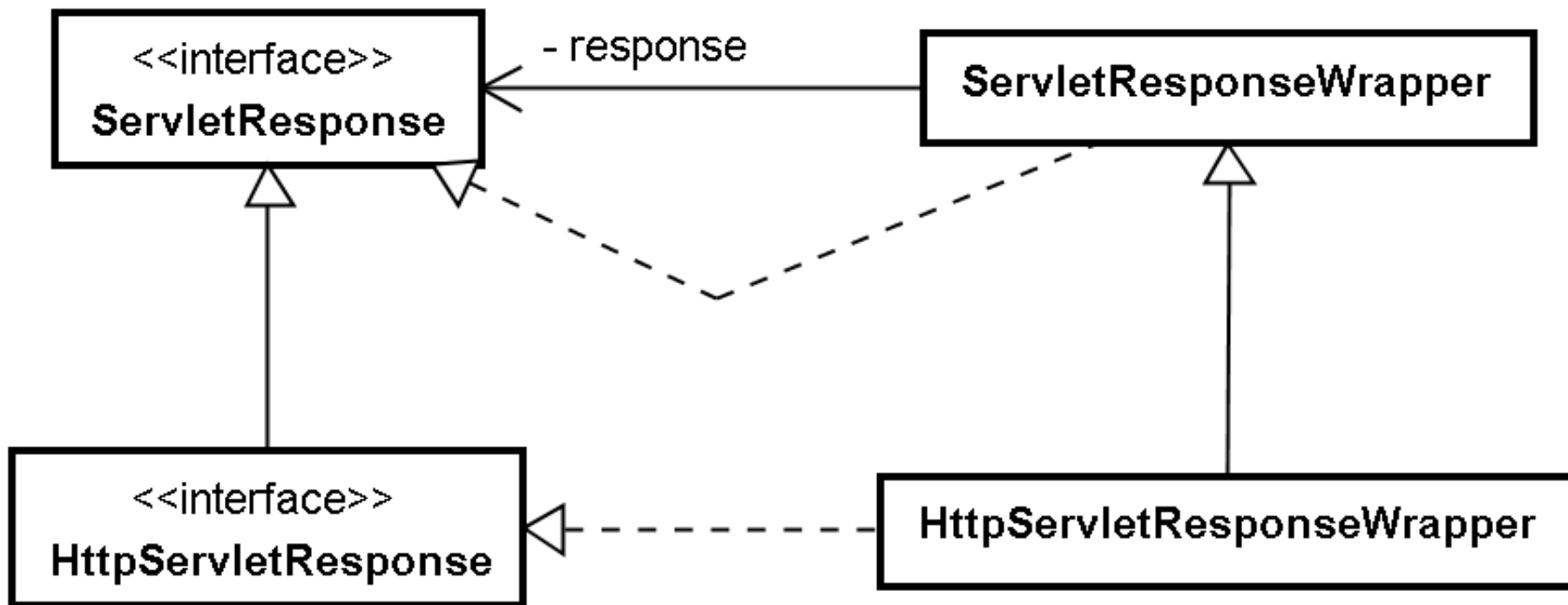
    public EncodingWrapper(HttpServletRequest request, String ENCODING) {
        super(request);
        this.ENCODING = ENCODING;
    }

    @Override
    public String getParameter(String name) {
        return Optional.ofNullable(getRequest().getParameter(name))
            .map(this::toEncoding)
            .orElse(null);
    }

    private String toEncoding(String original) {
        String value;
        try {
            byte[] b = original.getBytes("ISO-8859-1");
            value = new String(b, ENCODING);
        } catch (UnsupportedEncodingException e) {
            throw new RuntimeException(e);
        }
        return value;
    }
}
```

```
@WebFilter(  
    urlPatterns = { "/"* } ,  
    initParams = { @WebInitParam(name = "ENCODING", value = "UTF-8") }  
)  
public class Encoding extends HttpFilter {  
    public void doFilter(  
        HttpServletRequest request, HttpServletResponse response, FilterChain chain)  
        throws IOException, ServletException {  
        String encoding = getInitParameter("ENCODING");  
        HttpServletRequest req = request;  
        if ("GET".equals(request.getMethod())) {  
            req = new EncodingWrapper(req, encoding);  
        } else {  
            req.setCharacterEncoding(encoding);  
        }  
        chain.doFilter(req, response);  
    }  
}
```

回應包裹器



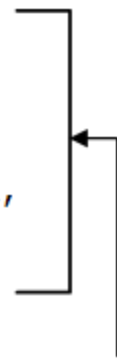
實作壓縮過濾器

```
public class CompressionWrapper extends HttpServletResponseWrapper {
    private GZipServletOutputStream gzServletOutputStream;
    private PrintWriter printWriter;

    public CompressionWrapper(HttpServletResponse response) {
        super(response);
    }

    @Override
    public ServletOutputStream getOutputStream() throws IOException {
        if(printWriter != null) { ← ❶ 已呼叫過 getWriter()，再呼叫
            throw new IllegalStateException();           getOutputStream() 就丟出例外
        }
        if (gzServletOutputStream == null) {
            gzServletOutputStream =
                new GZipServletOutputStream(getResponse().getOutputStream());
        }
        return gzServletOutputStream; ❷ 建立有壓縮功能的
    }                                     GzipServletOutputStream
                                         物件
}
```

```
public PrintWriter getWriter() throws IOException {  
    if(gzServletOutputStream != null) { ← ❸ 已呼叫過 getOutputStream() ,  
        throw new IllegalStateException();           再呼叫 getWriter() 就丟出例外  
    }  
    if (printWriter == null) {  
        gzServletOutputStream =  
            new GZipServletOutputStream(  
                getResponse().getOutputStream());  
        OutputStreamWriter osw =  
            new OutputStreamWriter(gzServletOutputStream,  
                getResponse().getCharacterEncoding());  
        printWriter = new PrintWriter(osw);  
    }  
    return printWriter;  
}
```



❹ 建立 GzipServletOutputStream 物件，供建構 PrintWriter 時使用

```
@Override
public void flushBuffer() throws IOException {
    if(this.printWriter != null) {
        this.printWriter.flush();
    }
    else if(this.gzServletOutputStream != null) {
        this.gzServletOutputStream.flush();
    }
    super.flushBuffer();
}
```

```
public void finish() throws IOException {
    if(this.printWriter != null) {
        this.printWriter.close();
    }
    else if(this.gzServletOutputStream != null) {
        this.gzServletOutputStream.finish();
    }
}
```

```
@Override
public void setContentLength(int len) {}
```

```
@Override
public void setContentLengthLong(long length) {}
```

```
}
```

← ❹ 不實作方法內容，因為真正的輸出會被壓縮

```
@WebFilter("/*")
public class Compression extends HttpFilter {
    protected void doFilter(
        HttpServletRequest request, HttpServletResponse response,
        FilterChain chain) throws IOException, ServletException {

        String encodings = request.getHeader("Accept-Encoding");
        if (encodings != null && encodings.contains("gzip")) { ← ❶ 檢查是否接受
                                                                gzip 壓縮格式

            CompressionWrapper responseWrapper =
                new CompressionWrapper(response); ← ❷ 建立回應包裹器
            responseWrapper.setHeader("Content-Encoding", "gzip"); ←
                                                                    ❸ 設定回應內容編碼
                                                                    為 gzip 格式

            chain.doFilter(request, responseWrapper); ←
                                                                ❹ 下一個過濾器

            responseWrapper.finish(); ← ❺ 呼叫 GZIPOutputStream 的
                                                                    finish()方法完成壓縮輸出
        }
        else {
            chain.doFilter(request, response); ← ❻ 不接受壓縮，直接進行
                                                                    下一個過濾器
        }
    }
}
```

簡介AsyncContext

- 在Servlet 3.0中，在ServletRequest上提供了**startAsync()**方法

```
AsyncContext startAsync() throws java.lang.IllegalStateException;  
AsyncContext startAsync(ServletRequest servletRequest,  
                        ServletResponse servletResponse)  
                        throws java.lang.IllegalStateException
```

- 呼叫了startAsync()方法取得AsyncContext物件之後，此次請求的回應會被延後，並釋放容器分配的執行緒

簡介AsyncContext

- `getRequest()`
- `getResponse()`
- `complete()`
- `dispatch()`

簡介AsyncContext

- 必須告知容器此Servlet支援非同步處理

```
@WebServlet(urlPatterns = "/asyncXXX", asyncSupported = true)
public class AsyncServlet extends HttpServlet {
    ...
```

```
    ...
    <servlet>
        <servlet-name>AsyncXXX</servlet-name>
        <servlet-class>cc.openhome.AsyncXXX</servlet-class>
        <async-supported>true</async-supported>
    </servlet>
    ...
```

簡介AsyncContext

- 若其前端有過濾器，則過濾器亦需標示其支援非同步處理

```
@WebFilter(urlPatterns = "/asyncXXX", asyncSupported = true)
public class AsyncFilter extends HttpFilter{
    ...

    ...
    <filter>
        <filter-name>AsyncFilter</filter-name>
        <filter-class>cc.openhome.AsyncFilter</filter-class>
        <async-supported>true</async-supported>
    </filter>
    ...
}
```

```
@WebServlet(  
    urlPatterns={"/async"},  
    asyncSupported = true ← ❶ 標註 Servlet 支援非同步處理  
)  
public class AsyncServlet extends HttpServlet {  
    protected void doGet(  
        HttpServletRequest request, HttpServletResponse response)  
        throws ServletException, IOException {  
        response.setContentType("text/html; charset=UTF8");  
        AsyncContext ctx = request.startAsync(); ← ❷ 開始非同步處理，會在  
                                                    service() 完成後釋放執行緒  
        ❸ 建立 CompletableFuture  
        ↓  
        doAsync(ctx).thenApplyAsync(String::toUpperCase) ← ❹ 處理結果轉大寫  
        .thenAcceptAsync(resource -> {  
            ❺ 輸出結果  
            ↓  
            try {  
                ctx.getResponse().getWriter().println(resource);  
                ctx.complete(); ← ❻ 對瀏覽器完成回應  
            } catch (IOException e) {  
                throw new UncheckedIOException(e);  
            }  
        })  
    }  
}
```

```
private CompletableFuture<String> doAsync(AsyncContext ctx) {  
    return CompletableFuture.supplyAsync(() -> {  
        try {  
            String resource = ctx.getRequest().getParameter("resource");  
            Thread.sleep(10000); ← ⑦ 模擬冗長請求  
            return String.format("%s back finally...XD", resource);  
        } catch (InterruptedException e) {  
            throw new RuntimeException(e);  
        }  
    });  
}
```

應用

- 非同步Long Polling
- 非同步Server-Sent Event
- 搭配ReadListener
- 搭配WriteListener

綜合練習 / 微網誌

- 先停下來檢討目前的應用程式，有哪些維護上的問題
- 在不改變目前應用程式的功能下，程式碼必須作出哪些調整
- 讓每個程式碼職責上變得更為清晰，對於將來的維護更有幫助